

Guide 3

Machine Learning Experiment Tracking and Management

2025/2026

1 Introduction

This guide introduces TensorBoard and MLflow for monitoring and visualizing training metrics for Deep Learning models. Next, you will work with the *MNIST* dataset and log metrics like training loss, test accuracy, and system metrics (CPU and memory usage).

The main goals are:

- Understand how to configure TensorBoard and MLflow with PyTorch.
- Monitor the training process in real-time.
- Compare training time, CPU and memory usage, and other system metrics.
- Run controlled tests and compare hyperparameter impact across runs.

2 Project Structure

This guide assumes the following tree layout:

```
guide3/  
  create_env.sh  
  requirements_arm.txt  
  pytorchtracking.py  
  submit_pytorchtracking.sh
```

Directory navigation to reach these files:

```
cd /projects/F202500010HPCVLABUMINHO/guides/guide3
```

Recommended workflow: copy to your personal folder and work there.

```
cd /path/to/your/personal/folder  
cp -r /projects/F202500010HPCVLABUMINHO/guides/guide3 .
```

3 Setup

Use the existing script `create_env.sh` with `sbatch` to create a virtual environment and install dependencies in `dev-arm`. The dependencies are listed in `requirements_arm.txt`.

```
torch
torchvision
tensorflow
tensorboard
tensorboardX
mlflow
psutil
```

Submit the environment creation job with:

```
cd /path/to/your/personal/folder/guide3
sbatch create_env.sh
```

```
#!/bin/bash
#SBATCH --job-name=create_env
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=46
#SBATCH --time=00:30:00
#SBATCH --account=<your_account>
#SBATCH --partition=dev-arm
#SBATCH --output=install.out
#SBATCH --error=install.err

module purge
module load Python/3.12.3-GCCcore-13.3.0

python -m venv /<SOURCE/TO/YOUR/ENV>
source /<SOURCE/TO/YOUR/ENV>/bin/activate

pip install --upgrade pip
pip install -r requirements_arm.txt

echo "Env created."
```

3.1 Dataset

The script uses `datasets.MNIST('data', ..., download=True)`, so MNIST is downloaded automatically to your local `data/` folder (inside your personal guide3 copy) if it is not already available. Keep using a partition with internet access (`dev-arm`) for the first run.

4 MLflow

Before implementing the model, we will configure MLflow to track, log, and manage experiments, including training metrics, parameters, and system-level statistics.

In this setup, the training script uses a local SQLite backend store file (defined in the code with `mlflow.set_tracking_uri()`). To visualize those runs in *OpenOnDemand* MLflow, launch the MLflow app and point the backend store to that same SQLite database file.

1. Go to the Deucalion *OpenOnDemand* web portal (<https://docs.macc.fccn.pt/openondemand/>).
2. Launch an MLflow job from the available apps.
3. In the form, specify the runtime (in hours), the queue (normal-x86), the account, and the backend store option `--backend-store-uri sqlite:///path/to/your/mlflow.db` to define where MLflow stores results.
4. Once running, click `Connect to MLflow`.

5 PyTorch Model

5.1 Check the `pytorchtracking.py`

This script trains a simple convolutional neural network on MNIST using only the CPU and logs metrics to both TensorBoard and MLflow.

You may notice that this script:

- Uses the SummaryWriter to log scalars and histograms.
- Uses MLflow for experiment tracking.
- Visualizes the model computation graph and tracks system metrics (such as CPU and memory usage) using psutil.

Make sure the dataset directory and the virtual environment are accessible.

5.2 Create bash script `submit_pytorchtracking.sh`

Then, submit a bash script to run your training job.

```
#!/bin/bash
#SBATCH --job-name=pytorchtracking
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=46
#SBATCH --time=4:00:00
#SBATCH --account=<your_account>
#SBATCH --partition=dev-arm
#SBATCH --output=job_%j.out
#SBATCH --error=job_%j.err

module purge
module load Python/3.12.3-GCCcore-13.3.0

source /<SOURCE/TO/YOUR/ENV>/bin/activate

python3 pytorchtracking.py

Submit with:

cd /path/to/your/personal/folder/guide3
sbatch submit_pytorchtracking.sh
```

6 TensorBoard

After training starts, you can visualize logs using TensorBoard. We will also use *OpenOnDemand* to launch a TensorBoard job.

1. Go to *OpenOnDemand*.
2. Navigate to the Interactive Apps section and select TensorBoard.
3. In the form, specify the runtime (in hours), the queue (normal-x86), the account, and the TensorBoard logdir to the `/path/to/your/tensorboard_runs/` directory generated by your script.
4. Submit the job.

5. Once it is running, click the `Connect to Tensorboard` button to open the TensorBoard interface in your browser.

Now, you can compare multiple runs, check training and test accuracy per epoch, and track system usage metrics.

7 Testing and Benchmarking

Use this section to define your own experiments by changing the training arguments.

7.1 Parameters you can change

The Python script supports the following parameters:

- `--epochs`
- `--batch-size`
- `--test-batch-size`
- `--lr`
- `--momentum`
- `--seed`
- `--log-interval`

In the current `submit_pytorchtracking.sh`, the command runs with default values:

```
python3 pytorchtracking.py
```

To test different settings, edit that line in the submit script, for example:

```
python3 pytorchtracking.py --epochs <E> --batch-size <B> --lr <LR> --momentum <M> --seed <S>
```

7.2 Metrics to collect from MLflow and TensorBoard

For each run, extract the following metrics:

7.2.1 TensorBoard metrics (via browser or MLflow UI):

- `test/accuracy`: final test accuracy at the last epoch.
- `test/loss`: final test loss at the last epoch.

7.2.2 MLflow system metrics (via MLflow UI or runs details):

- `system/cpu_utilization_percentage`: average CPU usage during the run.
- `system/system_memory_usage_megabytes`: peak memory usage.
- `system/disk_available_megabytes`: available disk space.

You can view these metrics in:

1. **TensorBoard**: open the TensorBoard UI and navigate to scalars for `test/accuracy` and `test/loss`.
2. **MLflow UI**: open MLflow, select your experiment, click on each run, and scroll to *System Metrics* section.

7.3 Benchmark table

Fill the table with your own tests. Record the final values at the last epoch.

Table 1: PyTorch Tracking Benchmark (MNIST)

Run	Epochs	Batch	LR	Test Acc.	Test Loss	Avg CPU	Max Mem (MB)
-	-	-	-	-	-	-	-

8 Questions

- What are the trade-offs between accuracy and resource consumption (CPU, memory)?
- Which parameter had the largest impact on final accuracy and convergence speed in your tests?
- What insights can you get by watching loss curves and accuracy plots during training?
- What differences did you notice between TensorBoard and MLflow regarding usability and features?
- Which system metrics could be useful to monitor for larger models?

9 Learning Outcomes

This guide demonstrates how to train and monitor a PyTorch model on ARM nodes, using TensorBoard and MLflow to log and visualize the computation graph and model parameters and track system resources.